Lab Assignment 4: Visualizing Machine Learning Models using Decision Trees

Objective: Train a classification model and interpret its logic through decision tree visualization.
Dataset: Iris (150 flowers, 4 features, 3 classes) from `sklearn.datasets`.
Tasks: (1) 70/30 train-test split and DecisionTreeClassifier training; (2) visualize the tree with `sklearn.tree.plot_tree`; (3) customize with filled=True, feature_names, class_names; (4) identify the top 2 features and the root node split.
Result: Test accuracy 97.78%. Top 2 features: **petal length (cm)** (0.551) and **petal width (cm)** (0.449). Root node split: **petal length (cm) <= 2.45** (gini = 0.667, 105 samples), which separates all setosa flowers.
Run with: `python lab4_decision_tree_visualization.py`

1. Python Code

```
# ======================================================================
# Lab Assignment 4: Visualizing Machine Learning Models using
#                   Decision Trees
# ----------------------------------------------------------------------
# Objective : Train a classification model and interpret its logic
#             through decision tree visualization.
# Dataset   : Iris (sklearn.datasets.load_iris)
# Run with  : python lab4_decision_tree_visualization.py
# Requires  : pip install scikit-learn pandas matplotlib
# Output    : Step-by-step console log + PNG plots in ./plots/
# ======================================================================

import os
from datetime import datetime

import matplotlib
import matplotlib.pyplot as plt
import pandas as pd
import sklearn
from sklearn.datasets import load_iris
from sklearn.metrics import (ConfusionMatrixDisplay, accuracy_score,
                             classification_report, confusion_matrix)
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree


# ---------- Helper: print a clearly visible step header --------------
def step(n, title):
    print("\n" + "=" * 70)
    print(f"STEP {n} : {title}")
    print("=" * 70)


OUT_DIR = "plots"
os.makedirs(OUT_DIR, exist_ok=True)


def save_plot(fig, file):
    path = os.path.join(OUT_DIR, file)
    fig.savefig(path, dpi=110, bbox_inches="tight")
    print(">> Plot saved to:", path)
    if matplotlib.get_backend().lower() != "agg":
        plt.show()
    plt.close(fig)


pd.set_option("display.width", 110)
pd.set_option("display.max_columns", None)
print("Lab Assignment 4 - Visualizing ML Models using Decision Trees")
print("Python libraries : scikit-learn", sklearn.__version__, "| pandas", pd.__version__,
      "| matplotlib", matplotlib.__version__)
print("Run on           :", datetime.now().strftime("%Y-%m-%d %H:%M:%S"))


# ======================================================================
```

```
# STEP 1 : Load the dataset
# ======================================================================
step(1, "Load the Iris dataset")

iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name="species")
feature_names = list(iris.feature_names)
class_names = list(iris.target_names)

print(">> Dataset loaded successfully.")
print(">> Shape of features X:", X.shape, "| Shape of target y:", y.shape)
print(">> Feature names:", feature_names)
print(">> Class names  :", class_names)

print("\n>> First 5 rows:")
print(X.assign(species=[class_names[i] for i in y]).head())

print("\n>> Class distribution:")
print(pd.Series([class_names[i] for i in y]).value_counts().rename("count"))


# ======================================================================
# STEP 2 : Train/test split and model training
# ======================================================================
step(2, "Split data (70% train / 30% test) and train DecisionTreeClassifier")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y)
print(">> train_test_split(test_size=0.30, random_state=42, stratify=y)")
print(">> Training set size:", X_train.shape[0], "rows")
print(">> Testing set size :", X_test.shape[0], "rows")

model = DecisionTreeClassifier(criterion="gini", max_depth=3, random_state=42)
model.fit(X_train, y_train)
print("\n>> Model trained:", model)
print(">> Tree depth      :", model.get_depth())
print(">> Number of leaves:", model.get_n_leaves())

y_pred = model.predict(X_test)
print(f"\n>> Training accuracy: {accuracy_score(y_train, model.predict(X_train)):.4f}")
print(f">> Testing accuracy : {accuracy_score(y_test, y_pred):.4f}")

print("\n>> Confusion matrix (rows = actual, columns = predicted):")
print(pd.DataFrame(confusion_matrix(y_test, y_pred),
                   index=class_names, columns=class_names))

print("\n>> Classification report:")
print(classification_report(y_test, y_pred, target_names=class_names))

fig, ax = plt.subplots(figsize=(5.5, 4.5))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, display_labels=class_names,
                                        cmap="Blues", ax=ax, colorbar=False)
ax.set_title("Confusion Matrix (Test Set)", fontweight="bold")
save_plot(fig, "1_confusion_matrix.png")


# ======================================================================
# STEP 3 : Visualize the fitted decision tree (customized plot_tree)
# ======================================================================
step(3, "Visualize the Decision Tree with sklearn.tree.plot_tree")

print(">> plot_tree parameters used:")
print("   filled=True, rounded=True, feature_names=feature_names,")
print("   class_names=class_names, impurity=True, proportion=False, fontsize=10")

fig, ax = plt.subplots(figsize=(16, 9))
plot_tree(model,
          feature_names=feature_names,
          class_names=class_names,
          filled=True,
          rounded=True,
          impurity=True,
          proportion=False,
          fontsize=10,
          ax=ax)
```

```
ax.set_title("Decision Tree Classifier - Iris Dataset (max_depth = 3, criterion = gini)",
             fontsize=15, fontweight="bold")
save_plot(fig, "2_decision_tree.png")

print("\n>> The same tree as text decision rules (export_text):")
print(export_text(model, feature_names=feature_names, class_names=class_names, decimals=2))


# ======================================================================
# STEP 4 : Feature importance and root node split
# ======================================================================
step(4, "Top 2 most important features and root node splitting criteria")

importance = (pd.Series(model.feature_importances_, index=feature_names)
              .sort_values(ascending=False))
print(">> Feature importances (Gini importance, sums to 1):")
print(importance.round(4))

top2 = importance.index[:2].tolist()
print(f"\n>> Top 2 most important features: 1) {top2[0]}   2) {top2[1]}")

tree = model.tree_
root_feature = feature_names[tree.feature[0]]
root_threshold = tree.threshold[0]
root_samples = tree.n_node_samples[0]
root_gini = tree.impurity[0]
left, right = tree.children_left[0], tree.children_right[0]

print("\n>> Root node splitting criteria:")
print(f"   Split rule : {root_feature} <= {root_threshold:.2f}")
print(f"   Criterion  : gini  (root gini = {root_gini:.3f})")
counts = (tree.value[0][0] * root_samples).round().astype(int)
print(f"   Samples    : {root_samples}  ("
      + ", ".join(f"{c} = {n}" for c, n in zip(class_names, counts)) + ")")
print(f"   True  branch -> {tree.n_node_samples[left]} samples, gini = "
      f"{tree.impurity[left]:.3f}, predicts '{class_names[tree.value[left][0].argmax()]}'")
print(f"   False branch -> {tree.n_node_samples[right]} samples, gini = "
      f"{tree.impurity[right]:.3f}, versicolor + virginica mixed, splits further")

fig, ax = plt.subplots(figsize=(7, 4))
colors = ["#2b8cbe" if f in top2 else "#bdbdbd" for f in importance.index]
importance[::-1].plot(kind="barh", color=colors[::-1], edgecolor="black", ax=ax)
for i, v in enumerate(importance[::-1]):
    ax.text(v + 0.01, i, f"{v:.3f}", va="center")
ax.set_title("Feature Importance (top 2 highlighted)", fontweight="bold")
ax.set_xlabel("Gini importance")
ax.set_xlim(0, importance.max() * 1.15)
save_plot(fig, "3_feature_importance.png")

print(f"\n>> Interpretation: the root node separates all setosa flowers with a single "
      f"rule\n   ({root_feature} <= {root_threshold:.2f}). The remaining splits use mainly "
      f"{top2[0]}\n   and {top2[1]} to separate versicolor from virginica.")


# ======================================================================
# Done
# ======================================================================
step(5, "Lab Assignment 4 completed")
print("Files generated:")
for f in sorted(os.listdir(OUT_DIR)):
    print("  -", os.path.join(OUT_DIR, f))
```

2. Execution Output (console printout)

```
Lab Assignment 4 - Visualizing ML Models using Decision Trees
Python libraries : scikit-learn 1.9.1 | pandas 3.0.5 | matplotlib 3.11.2
Run on           : 2026-10-26 21:32:23

======================================================================
STEP 1 : Load the Iris dataset
======================================================================
>> Dataset loaded successfully.
>> Shape of features X: (150, 4) | Shape of target y: (150,)
>> Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
>> Class names  : [np.str_('setosa'), np.str_('versicolor'), np.str_('virginica')]

>> First 5 rows:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm) species
0                5.1               3.5                1.4               0.2  setosa
1                4.9               3.0                1.4               0.2  setosa
2                4.7               3.2                1.3               0.2  setosa
3                4.6               3.1                1.5               0.2  setosa
4                5.0               3.6                1.4               0.2  setosa

>> Class distribution:
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64

======================================================================
STEP 2 : Split data (70% train / 30% test) and train DecisionTreeClassifier
======================================================================
>> train_test_split(test_size=0.30, random_state=42, stratify=y)
>> Training set size: 105 rows
>> Testing set size : 45 rows
>> Model trained: DecisionTreeClassifier(max_depth=3, random_state=42)
>> Tree depth      : 3
>> Number of leaves: 5

>> Training accuracy: 0.9810
>> Testing accuracy : 0.9778
>> Confusion matrix (rows = actual, columns = predicted):
            setosa  versicolor  virginica
setosa          15           0          0
versicolor       0          14          1
virginica        0           0         15

>> Classification report:
              precision    recall  f1-score   support
      setosa       1.00      1.00      1.00        15
  versicolor       1.00      0.93      0.97        15
   virginica       0.94      1.00      0.97        15

    accuracy                           0.98        45
   macro avg       0.98      0.98      0.98        45
weighted avg       0.98      0.98      0.98        45
>> Plot saved to: plots/1_confusion_matrix.png

======================================================================
STEP 3 : Visualize the Decision Tree with sklearn.tree.plot_tree
======================================================================
>> plot_tree parameters used:
   filled=True, rounded=True, feature_names=feature_names,
   class_names=class_names, impurity=True, proportion=False, fontsize=10
>> Plot saved to: plots/2_decision_tree.png

>> The same tree as text decision rules (export_text):
|--- petal length (cm) <= 2.45
|   |--- class: setosa
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.55
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: versicolor
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- class: virginica
|   |--- petal width (cm) >  1.55
|   |   |--- petal width (cm) <= 1.70
```

```
|   |   |   |--- class: versicolor
|   |   |--- petal width (cm) >  1.70
|   |   |   |--- class: virginica


======================================================================
STEP 4 : Top 2 most important features and root node splitting criteria
======================================================================
>> Feature importances (Gini importance, sums to 1):
petal length (cm)    0.5509
petal width (cm)     0.4491
sepal width (cm)     0.0000
sepal length (cm)    0.0000
dtype: float64

>> Top 2 most important features: 1) petal length (cm)   2) petal width (cm)

>> Root node splitting criteria:
   Split rule : petal length (cm) <= 2.45
   Criterion  : gini  (root gini = 0.667)
   Samples    : 105  (setosa = 35, versicolor = 35, virginica = 35)
   True  branch -> 35 samples, gini = 0.000, predicts 'setosa'
   False branch -> 70 samples, gini = 0.500, versicolor + virginica mixed, splits further
>> Plot saved to: plots/3_feature_importance.png

>> Interpretation: the root node separates all setosa flowers with a single rule
   (petal length (cm) <= 2.45). The remaining splits use mainly petal length (cm)
   and petal width (cm) to separate versicolor from virginica.

======================================================================
STEP 5 : Lab Assignment 4 completed
======================================================================
Files generated:
  - plots/1_confusion_matrix.png
  - plots/2_decision_tree.png
  - plots/3_feature_importance.png
```

3. Output Plots

Figure 1: Customized Decision Tree (plot_tree, filled=True, feature_names, class_names)

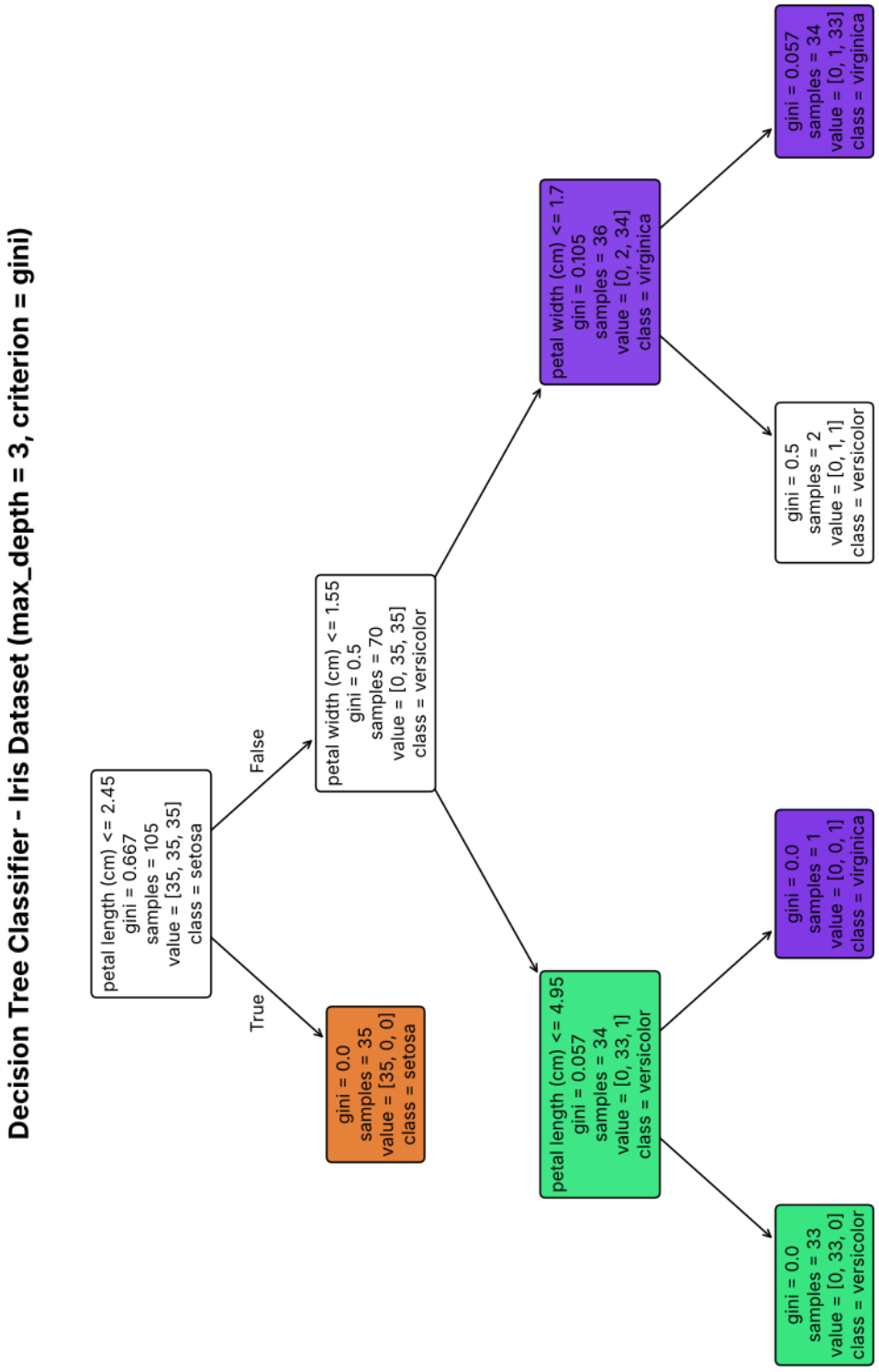


Figure 2: Feature Importance (top 2 highlighted)

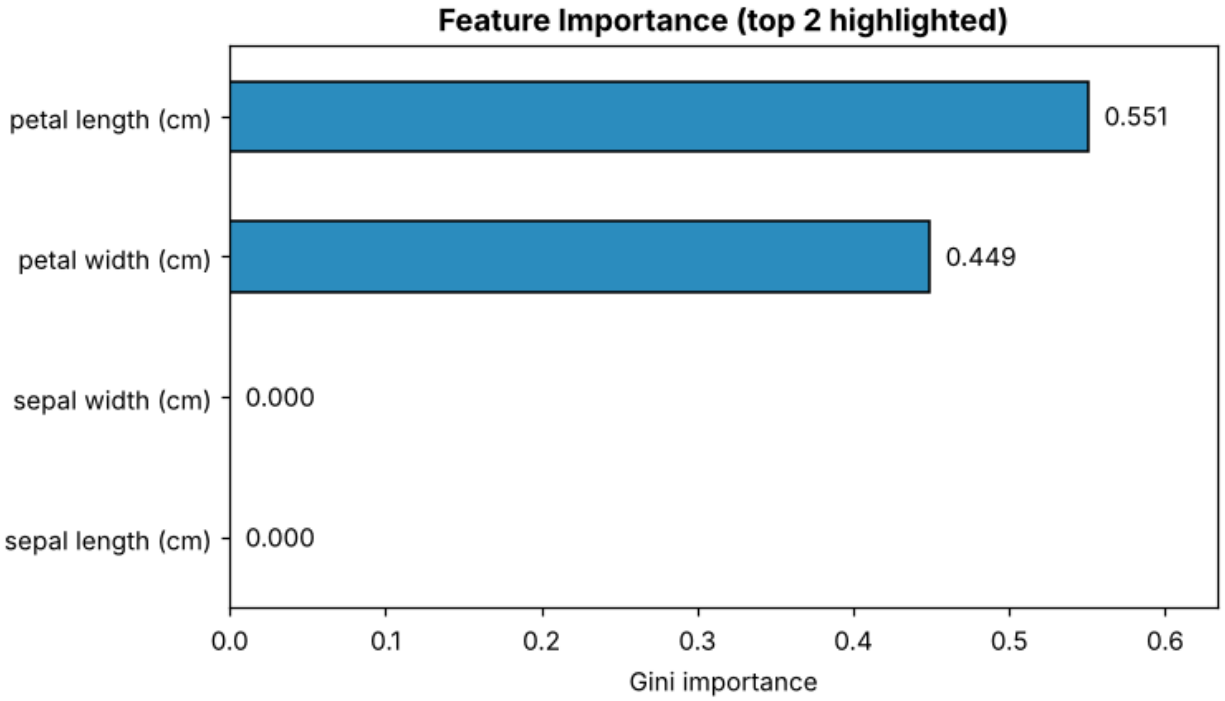


Figure 3: Confusion Matrix on the Test Set

